

2017-2018 → kst1

- ①. UNIX shell → display lines in a file a.txt that contain words starting with capital letters.

grep -E '\<[A-Z]' a.txt

start of word

- ② UNIX shell command $\rightarrow$ inverts all pairs of neighbouring digits in a file.
(a.txt).
ex: 123456 $\rightarrow$ 321654

ex: $a3972b \rightarrow a9327b$

save result temporarily

sed -E 's/([0-9])([0-9])/211/g' a.txt > temp.txt && mv temp.txt a.txt

substitute ← group1 group2 swap the 2 groups do it for all pairs on the line overwrite original file

execute next command
($\Rightarrow$) previous succeeded

- ③ Unix shell cmd $\rightarrow$ display for each line the sum of its numbers (a.txt contains on each line 2 numbers separated by space). ($\Rightarrow$ previous)

awk '{print \$1 + \$2 }' a.txt

↓
executed for (v) line

↓
address
of the
field 1

↓
imped file.

- ④ Display only unique lines (not duplicated from the start)

sort a.txt | uniq -u 1 → print only lines that appear once

for

a
a
b
b
c

, `uniq` would print

a
b
c

, but `uniq -u` would print

c

`uniq -c` would print

2
2
1

`uniq -c` would print

2	a
2	b
1	c

- ⑤ Unix shell script displaying the name of each ~~text~~ file in the current dir that contains the word "cat".

```
for f in *.txt; do
    grep -l "cat" "$f" && echo "$f"
done
```

quiet mode
(don't print)

execute next
command $\Leftrightarrow$ previous succeeded.

Better: (catalogue is not watched):

```
for f in *.txt; do
    grep -Eg '\<cat\>' "$f" && echo "$f"
done
```

start of word end of word

\$f\$ vs "\$f\$":

$f = \text{"my file.txt"}$
 $\text{cat } \$f \rightarrow \text{cat my file.txt}$
 ↓ ↓
 arg_1 arg_2

cat "\$f" →
→ cat "my file.txt"
arg1

⑥ Mark which process executes each line: the Parent, the Child, or both.

✓

P	C	$K = \text{fork}();$
P	C	$\text{if } (K == 0)$
	C	$\text{printf}(...);$
P		else
P		$\text{printf}(...);$
P	C	$\text{printf}(...);$

✓ ⑦ How many processes will be created, excluding the parent?

! $\text{fork}(); \text{wait}(); \text{fork}(); \text{wait}(); \text{fork}();$

→ only if parent has children
 $\text{wait}() =$ pause parent until child finishes.

$\text{fork}():$ P
 ↓
 C₁

$\text{wait}():$ P waits for C₁; no new processes.

$\text{fork}();$ (P)
 ↓
 C₁ → C₃
 ↓
 C₂

$\text{wait}():$ no new.

$\text{fork}():$ (P)
 ↓
 C₁ → C₃ → C₄
 ↓
 C₅
 ↓
 C₂ → C₆
 ↓
 C₇

⇒ 7 processes.

$\text{fork}():$ P → $\text{wait}()$
 ↓
 C₁

$\text{fork}():$ P
 ↓
 C₁ → $\text{wait}()$
 ↓
 C₂ → C₃

$\text{fork}():$ P
 ↓
 C₁ → C₃ → C₄
 ↓
 C₅
 ↓
 C₂ → C₆
 ↓
 C₇

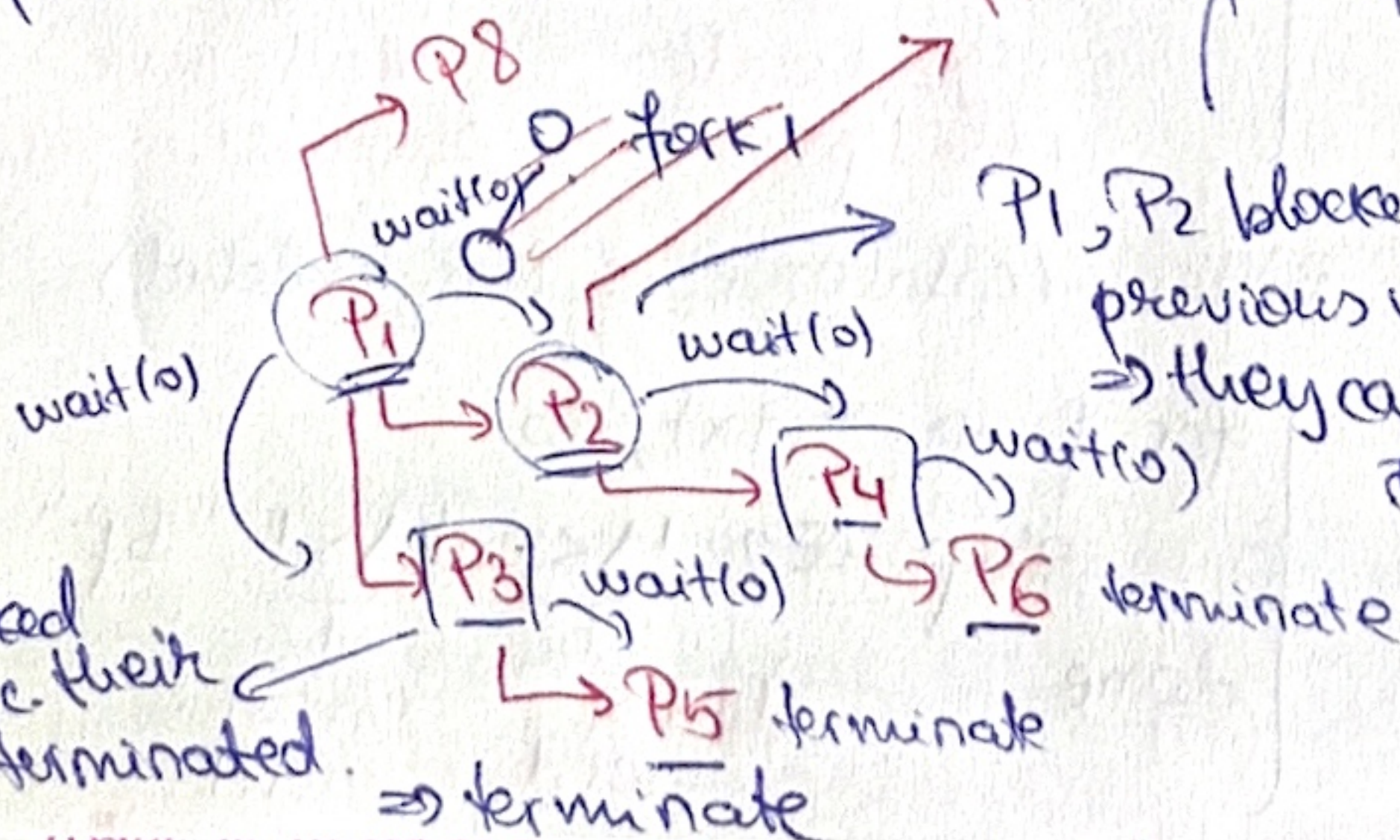
* Kiraaly Craft Stack Overflow:

$\text{fork}(); \text{fork}(); \text{wait}(); \text{fork}(); \text{wait}();$

3 $\text{forks}() \Rightarrow 2^3 = 8$ processes.

1 parent process } ⇒ 7 processes.

(P)
 ↓
 C₁ → C₃ → C₅
 ↓
 C₄
 ↓
 C₂ → C₆
 ↓
 C₇



console outputs possibilities (ignoring outputs by `exec`); when will they happen?

```
printf("A\n");  
exec(...);  
printf("B\n");
```

✓ if `exec(...)` succeeds $\Rightarrow$ output: A

if `exec(...)` fails $\Rightarrow$ output: A
B

↓
returns -1.

⑨ What does the system call `read()` when pipe is empty?

✓ if { pipe is empty
 $\exists$ at least one writer
 (one write end opened) } $\Rightarrow$ `read()` blocks ^{the process} until data becomes available. $\Rightarrow$ all processes will wait until there is something to read from the pipe; the `read()` call will only return when at least one byte of data is available.

if { pipe is empty
 every write end is closed } $\Rightarrow$ `read()` returns 0 (EOF).

⑩ What does `open()` do ~~into~~ before returning from opening a file?

✓ `open(..., O_RDONLY)` blocks until someone opens the FIFO for writing.

`open(..., O_WRONLY)` blocks until someone opens the FIFO for reading.

When opening a FIFO, `open()` may block before returning. A read-only open waits for a writer and a write-only open waits for a reader.

⑪ Threads over processes? Inter-thread better than inter-process, generally.

- efficiency: - threads are more efficient, lightweight than processes

- threads require fewer system resources than processes ✓

✓ - threads share the same-memory space } of the process that created them
 - file descriptors
 - other resources

$\Rightarrow$ lower overhead

$\Rightarrow$ faster communication

$\Rightarrow$ more efficient when multiple tasks need to be performed concurrently within a single program.

- shared memory: - threads within a process can access + modify shared memory.

$\Rightarrow$ efficient communication

$\Rightarrow$ good data-sharing between threads

$\Rightarrow$ no need for IPC mechanisms (complex)

$\Rightarrow$ sharing data between threads is simpler than sharing data between separate processes.

- sync: - threads can easily sync their execution through locks, semaphores, cond.var -

① coordinate activities; ② share resources safely; ③ avoid race conditions

③

- ⑫ "fa", "fb" are functions run in concurrent threads; $m = ?$ after the threads are finished?

```
pthread_mutex_t a, b;
int m = 0;
```

```
void* fa(void* p){
    pthread_mutex_lock(&a);
    m++;
    pthread_mutex_unlock(&a);
}
```

} protects m with mutex a

```
void* fb(void* p){
    pthread_mutex_lock(&b);
    m++;
    pthread_mutex_unlock(&b);
}
```

} protects same m with another mutex (b)

$a \neq b$

$m = 0$

fa reads 0

fb reads 0

fa writes 1

fb writes 1

- If fa() completes first $\Rightarrow m = 1$.
- If fb() completes second $\Rightarrow m = 2$

$\Rightarrow$ Value of m is undefined because the same shared variable is protected by different mutexes $\Rightarrow$ race condition. $\Rightarrow$ final value might be 1 or 2.

- ⑬ Schedule the jobs (Name / Duration / Deadline) so that they all meet their deadlines: A/5/9, B/7/13, C/1/10.

(SJF, STCF not needed)

Try CAB:
 C: finishes at 1; deadline: 10 $\Rightarrow 1 \leq 10 \Rightarrow \checkmark$
 A: finishes at $1+5=6$; deadline: 9 $\Rightarrow 6 \leq 9 \checkmark$
 B: finishes at $6+7=13$; deadline: 13 $\Rightarrow 13 \leq 13 \checkmark$

X ABC:
 A: finishes at $5 \leq 9 \checkmark$
 B: $5+7=12 \leq 13 \checkmark$
 C: $12+1=13 > 10 \times$

ACB:
 A: $5 \leq 9 \checkmark$
 C: $5+1=6 \leq 10 \checkmark$
 B: $6+7=13 \leq 10 \checkmark$

- $\Rightarrow$ B must be last (has largest deadline)
 $\Rightarrow$ CAB, ACB valid schedulings.

actually accesses the root of the mounted partition.

① (a) (b) (c) (d) (e) (f) (g) (h) (i) (j) (k) (l) (m) (n) (o) (p) (q) (r) (s) (t) (u) (v) (w) (x) (y) (z) (+)

- **external fragmentation** occurs when free memory blocks are scattered in the memory space $\Rightarrow$ inability to allocate contiguous memory blocks for larger segments/modules.

- demand paging (technique)

- page fault

- load on first access

Demond paging technique: load into memory $\Rightarrow$ pages are needed in the program;
when program starts $\Rightarrow$ OS loads initial set of pages

- ↳ program's entry point
- ↳ libraries
- ↳ shared code.

⑩ When does a process change state from RUN to READY?

- when it's **preempted**, meaning the CPU is given to another process

- causes: 1) Round Robin = time quantum expires

- 2) a higher-priority process becomes ready

! and i-modes having S direct link, one simple indirection link, one double indir. link, one triple indirection link, give formula for the max. file size possible.

S = no. of direct data blocks

A = ~~addresses~~ nr. of addresses inside one ~~simple~~ indirect block

$$A^2 = \text{nr. of data blocks reachable by double indirection}$$

$A^3 = 1-1-1$ triple indirection

B = block size in bytes. (bytes/data block)

$$2) \quad S + A + A^2 + A^3 \quad \Rightarrow \quad S * B + A * B + A^2 * B + A^3 * B =$$

$$\downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow$$

$$\text{block} \quad B \quad B \quad B \quad \text{mr. blocks} \quad \text{size} \quad \text{blocks} \quad \text{size} \quad = B * (S + A + A^2 + A^3)$$

$$\text{size} = B$$

! (18) What happens to the data when you delete a file having a hard link pointing to it?

- Deleting a file removes only that directory entry.
 - If $\exists$ hard link, the file data remains accessible because the inode reference count is > 0 . The data is deleted only when the last hard link is removed (inode reference count = 0)
- (see 21 Jun 2023)
(see CanvaTeodora on git)
- hard link $\rightarrow$ inode

(19) Prevent deadlocks.

(See CanvaTeodora for complete answer)

- acquire resources/mutexes in the same order. (prevents circular wait)
 - $\rightarrow$ resource allocation algorithms: - Banker's alg.
- resource ordering
- avoid holding $>$ resources simultaneously
 - $\rightarrow$ request/acquire one resource at a time (prevents circular dependencies)
- deadlock detection algorithm
 - $\rightarrow$ periodically checks for deadlocks and if it finds any $\Rightarrow$
 - $\Rightarrow$ process termination
 - $\Rightarrow$ resource preemption
- resource scheduling and allocation protocols

deadlocks.



(20) Binary Semaphore? Effect of its P method? when called by multiple concurrent processes/threads?

- $\Rightarrow$ Binary semaphore = semaphore that has only values 0/1.
 - = a synchronization primitive that can have two states: 0 (locked) and 1 (unlocked)
 - $\Rightarrow$ used to control access to a shared memory resource
 - coordinate execution of concurrent processes/thr.

$\Rightarrow$ P method = sem_wait $\Rightarrow$ decrements counter.

- $\rightarrow$ if value $\geq 1 \Rightarrow$ decrements $\Rightarrow$ continues execution.
- $\rightarrow$ if value = 0 $\Rightarrow$ decrements $\Rightarrow$ caller blocks until another process/thread performs V (sem-post)